

0/1 Knapsack by Branch and Bound

1 Problem Definition

0/1 Knapsack

Given n items with weights w_i and values v_i , and a knapsack of capacity W , choose the most valuable subset of items whose total weight does not exceed W .

Each item is either selected completely or not selected at all:

$$x_i \in \{0, 1\},$$

where

$$x_i = \begin{cases} 1, & \text{item } i \text{ is included,} \\ 0, & \text{item } i \text{ is excluded.} \end{cases}$$

The goal is to maximize

$$\sum_{i=1}^n v_i x_i$$

subject to

$$\sum_{i=1}^n w_i x_i \leq W.$$

2 Example Data

The items are ordered in decreasing value-to-weight ratio:

$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \dots \geq \frac{v_n}{w_n}.$$

Item	Weight w_i	Value v_i	v_i/w_i
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

The knapsack capacity is

$$W = 10.$$

3 State-Space Tree

The state-space tree is a binary tree. At level i , the algorithm decides whether to include or exclude item i .

left branch = include the next item, right branch = exclude the next item.

Each node stores:

$$(w, v, ub),$$

where

w = current total weight, v = current total value, ub = upper bound on possible value.

4 Upper Bound Formula

For a node after deciding the first i items, suppose the next undecided item is $i + 1$. A simple upper bound is:

Upper Bound

$$ub = v + (W - w) \left(\frac{v_{i+1}}{w_{i+1}} \right).$$

This bound says: keep the value v already selected, then fill the remaining capacity $W - w$ using the best value per unit weight among the remaining ordered items.

Why This Is an Upper Bound

The formula is optimistic because it acts as if the remaining capacity can be filled fractionally using the next best ratio. The real 0/1 solution cannot be better than this optimistic estimate.

5 Root Node

At the root, no item has been selected:

$$w = 0, \quad v = 0.$$

The next item is item 1, whose ratio is 10. Hence,

$$ub = 0 + (10 - 0)(10) = 100.$$

Root

$$w = 0, \quad v = 0, \quad ub = 100.$$

6 Branch on Item 1

From the root, create two branches.

Node	Weight	Value	Upper-bound calculation
With item 1	4	40	$40 + (10 - 4)(6) = 76$
Without item 1	0	0	$0 + (10 - 0)(6) = 60$

The node with item 1 has the larger upper bound:

$$\boxed{76 > 60.}$$

So the best-first branch-and-bound algorithm expands that node next.

7 Branch on Item 2

Now item 1 is included:

$$w = 4, \quad v = 40.$$

Node	Weight	Value	Status
With item 2	$4 + 7 = 11$	$40 + 42 = 82$	infeasible because $11 > 10$
Without item 2	4	40	$ub = 40 + (10 - 4)(5) = 70$

The branch that includes item 2 is discarded immediately because it exceeds the capacity.

8 Branch on Item 3

The best live node is now:

$$w = 4, \quad v = 40, \quad ub = 70.$$

This means item 1 is included and item 2 is excluded.

Node	Weight	Value	Upper bound or status
With item 3	$4 + 5 = 9$	$40 + 25 = 65$	$ub = 65 + (10 - 9)(4) = 69$
Without item 3	4	40	$ub = 40 + (10 - 4)(4) = 64$

The branch with item 3 is more promising because

$$69 > 64.$$

9 Branch on Item 4

Now items 1 and 3 are included, while item 2 is excluded:

$$w = 9, \quad v = 65.$$

Node	Weight	Value	Status
With item 4	$9 + 3 = 12$	$65 + 12 = 77$	infeasible because $12 > 10$
Without item 4	9	65	feasible complete subset

The branch without item 4 gives a feasible solution:

$$\boxed{\{1, 3\}}$$

with total weight

$$4 + 5 = 9$$

and total value

$$40 + 25 = 65.$$

So the current best value becomes

$$\text{bestValue} = 65.$$

10 Pruning Remaining Nodes

For a maximization problem, a node is pruned when its upper bound is not better than the best feasible value already found.

Pruning Rule

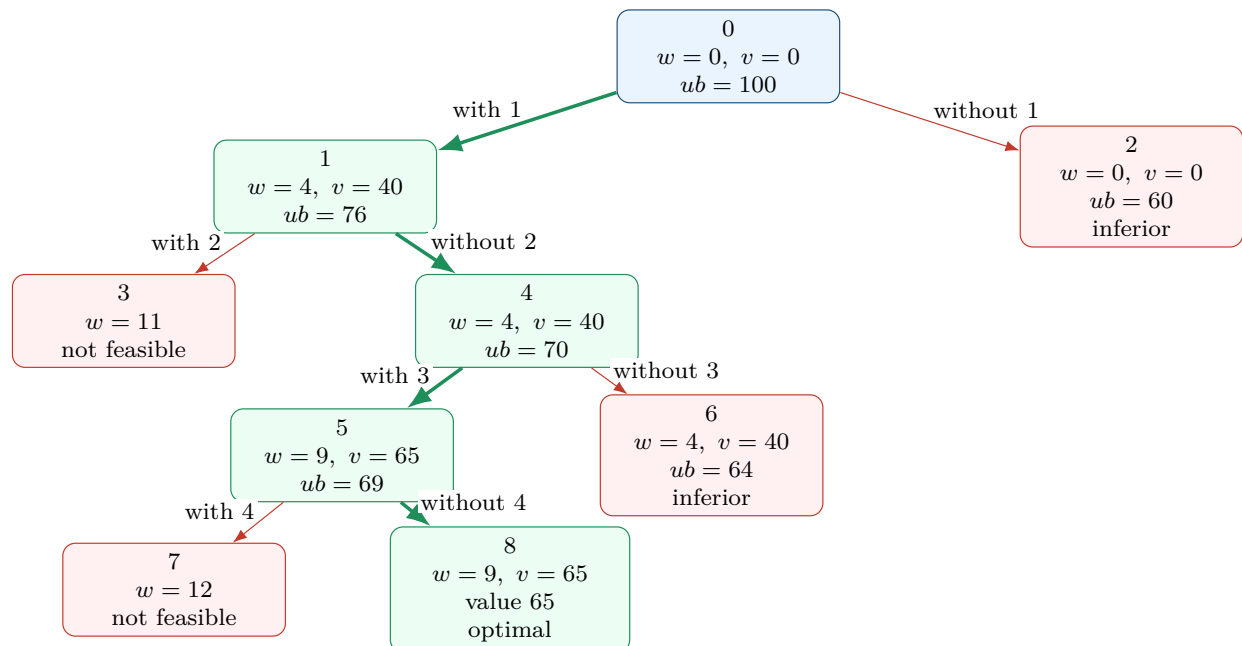
$$ub \leq \text{bestValue} \implies \text{prune the node.}$$

After finding $\text{bestValue} = 65$, the remaining live nodes are:

Node	Upper bound	Decision
Without item 1	60	prune
Without item 3 after choosing item 1	64	prune

Both nodes cannot improve the value 65, so they are terminated.

11 State-Space Tree Diagram



12 Optimal Solution

The optimal subset is:

$$\{1, 3\}.$$

The decision vector is:

$$(x_1, x_2, x_3, x_4) = (1, 0, 1, 0).$$

Total weight:

$$4 + 5 = 9 \leq 10.$$

Total value:

$$40 + 25 = 65.$$

Answer

$$\text{Maximum value} = 65.$$

13 Second Worked Problem

Now solve another 0/1 knapsack instance:

Original item	1	2	3	4
Value	10	10	12	18
Weight	2	4	6	9

with capacity

$$W = 15.$$

Sort by Value-to-Weight Ratio

Compute

$$\frac{v_i}{w_i}.$$

Original item	w_i	v_i	v_i/w_i
1	2	10	5
2	4	10	2.5
3	6	12	2
4	9	18	2

Items 3 and 4 have the same ratio. We keep the original order between them. Therefore, the ordered items are:

$$1, 2, 3, 4.$$

Root Bound

At the root,

$$w = 0, \quad v = 0.$$

The best ratio is item 1's ratio 5. So

$$ub = 0 + (15 - 0)(5) = 75.$$

Branch on Item 1

Node	Weight	Value	Upper-bound calculation
With item 1	2	10	$10 + (15 - 2)(2.5) = 42.5$
Without item 1	0	0	$0 + (15 - 0)(2.5) = 37.5$

The branch with item 1 is expanded first because

$$42.5 > 37.5.$$

Branch on Item 2

Now item 1 is included:

$$w = 2, \quad v = 10.$$

Node	Weight	Value	Upper-bound calculation
With item 2	$2 + 4 = 6$	$10 + 10 = 20$	$20 + (15 - 6)(2) = 38$
Without item 2	2	10	$10 + (15 - 2)(2) = 36$

The branch with item 2 is expanded next because

$$38 > 36.$$

Branch on Item 3

Now items 1 and 2 are included:

$$w = 6, \quad v = 20.$$

Node	Weight	Value	Upper-bound calculation
With item 3	$6 + 6 = 12$	$20 + 12 = 32$	$32 + (15 - 12)(2) = 38$
Without item 3	6	20	$20 + (15 - 6)(2) = 38$

Both branches have the same upper bound. Use a consistent tie rule; here we expand the branch with item 3 first.

Branch on Item 4 After Choosing Items 1, 2, and 3

Now

$$w = 12, \quad v = 32.$$

Node	Weight	Value	Status
With item 4	$12 + 9 = 21$	$32 + 18 = 50$	infeasible because $21 > 15$
Without item 4	12	32	feasible subset $\{1, 2, 3\}$

So the current best feasible value is

$$\boxed{\text{bestValue} = 32}$$

from subset

$$\boxed{\{1, 2, 3\}}.$$

Check the Other Tied Branch

The tied branch after item 3 was skipped is:

$$\{1, 2\}, \quad w = 6, \quad v = 20.$$

If item 4 is included:

$$w = 6 + 9 = 15, \quad v = 20 + 18 = 38.$$

This is feasible and improves the best value:

$$38 > 32.$$

Therefore,

$$\boxed{\text{bestValue} = 38}$$

with subset

$$\boxed{\{1, 2, 4\}}.$$

If item 4 is excluded, the value remains 20, which is worse.

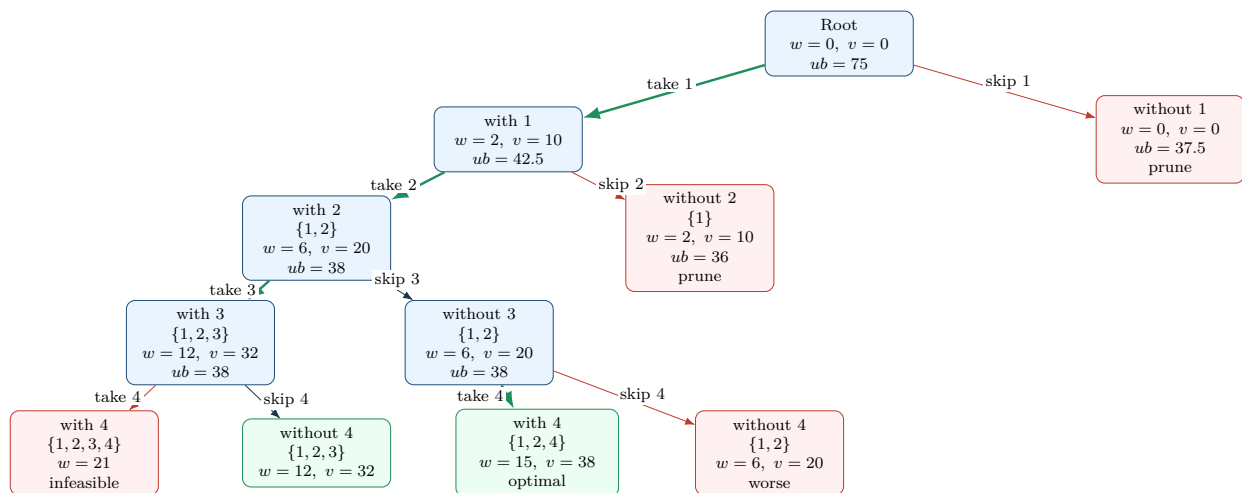
Prune Remaining Branches

The remaining live branches have upper bounds:

Branch	ub	Decision
Without item 1	37.5	prune because $37.5 < 38$
With item 1, without item 2	36	prune because $36 < 38$

No remaining branch can improve the current best value 38.

State-Space Tree for the Second Problem



Second Problem Answer

The optimal subset is

$$\{1, 2, 4\}.$$

Total weight:

$$2 + 4 + 9 = 15 \leq 15.$$

Total value:

$$10 + 10 + 18 = 38.$$

Second Problem Answer

Maximum value = 38.

14 Third Worked Problem

Now solve:

Original item	1	2	3	4
Weight	5	7	4	2
Value	30	28	20	24

with capacity

$$W = 12.$$

Sort by Value-to-Weight Ratio

Original item	w_i	v_i	v_i/w_i
1	5	30	6
2	7	28	4
3	4	20	5
4	2	24	12

So the branch-and-bound order is:

$$4, 1, 3, 2.$$

Sorted label	Original item	w	v	v/w
S_1	4	2	24	12
S_2	1	5	30	6
S_3	3	4	20	5
S_4	2	7	28	4

Item Numbering

From now on, branching uses the sorted labels S_1, S_2, S_3, S_4 . The final answer is converted back to the original item numbers.

Root and Main Branches

At the root,

$$w = 0, \quad v = 0, \quad ub = 0 + (12)(12) = 144.$$

Node	Weight	Value	Upper-bound calculation
With S_1	2	24	$24 + (12 - 2)(6) = 84$
Without S_1	0	0	$0 + (12)(6) = 72$

Expand the branch with S_1 , because

$$84 > 72.$$

Continue with S_2

Now S_1 is included:

$$w = 2, \quad v = 24.$$

Node	Weight	Value	Upper-bound calculation
With S_2	$2 + 5 = 7$	$24 + 30 = 54$	$54 + (12 - 7)(5) = 79$
Without S_2	2	24	$24 + (12 - 2)(5) = 74$

Expand the branch with S_2 , because

$$79 > 74.$$

Continue with S_3

Now S_1 and S_2 are included:

$$w = 7, \quad v = 54.$$

Node	Weight	Value	Upper-bound calculation
With S_3	$7 + 4 = 11$	$54 + 20 = 74$	$74 + (12 - 11)(4) = 78$
Without S_3	7	54	$54 + (12 - 7)(4) = 74$

Expand the branch with S_3 , because

$$78 > 74.$$

Decision on S_4

Now S_1, S_2, S_3 are included:

$$w = 11, \quad v = 74.$$

Node	Weight	Value	Status
With S_4	$11 + 7 = 18$	$74 + 28 = 102$	infeasible because $18 > 12$
Without S_4	11	74	feasible sorted subset $\{S_1, S_2, S_3\}$

So the current best value is

$$\text{bestValue} = 74$$

from original items

$$\{1, 3, 4\}.$$

Pruning

The remaining live branches cannot improve the best value:

Branch	<i>ub</i>	Decision
Without S_1	72	prune because $72 < 74$
With S_1 , without S_2	74	prune because $74 \leq 74$
With S_1, S_2 , without S_3	74	prune because $74 \leq 74$

Third Problem Answer

The optimal subset is

$$\{1, 3, 4\}.$$

Total weight:

$$5 + 4 + 2 = 11 \leq 12.$$

Total value:

$$30 + 20 + 24 = 74.$$

Third Problem Answer

$$\text{Maximum value} = 74.$$

15 Pseudocode

Best-First Knapsack Branch and Bound

```
KnapsackBranchAndBound(items, W):
    sort items by decreasing value/weight ratio
    root.weight = 0
    root.value = 0
    root.bound = upperBound(root)

    liveNodes = {root}
    bestValue = 0
    bestSubset = empty

    while liveNodes is not empty:
        node = remove live node with largest upper bound

        if node.bound <= bestValue:
            continue

        create child that includes the next item
```

```

    if child.weight <= W:
        if child.value > bestValue:
            bestValue = child.value
            bestSubset = child.subset
        child.bound = upperBound(child)
        if child.bound > bestValue:
            add child to liveNodes

    create child that excludes the next item
    child.bound = upperBound(child)
    if child.bound > bestValue:
        add child to liveNodes

return bestSubset and bestValue

```

16 Complexity

Each item creates two choices:

include or exclude.

Therefore, the worst-case search tree can still have

$$2^n$$

nodes.

Worst-case time = $\mathcal{O}(2^n)$.

Branch and Bound is useful because upper bounds often prune many nodes before the full tree is generated.

17 Final Review

0/1 Knapsack Branch and Bound Essentials

Order items by decreasing v_i/w_i .
Each node stores w , v , and ub .
 $ub = v + (W - w)(v_{i+1}/w_{i+1})$.
For maximization, prune when $ub \leq \text{bestValue}$.
Best subset here is $\{1, 3\}$ with value 65.